

Backend Schema

MaaSaathi — embedded JSON document database (server/database.js)

The backend does not use SQL tables. It uses a single JSON file (server/data/maasaathi_db.json) as a lightweight document store, with atomic writes on every change. Each user is a document keyed by user ID, containing nested arrays for appointments, records, symptoms, and more.

```
// Top-level database document
{
  version, created_at, updated_at,
  users: {
    [userId]: {
      id, profile: { name, status, edd, cycleLength,
        periodLength, lastPeriodDate, bloodGroup,
        doctorName, hospitalName, ec_name,
        ec_phone, city },
      appointments: [ { id, title, date, time,
        doctor, clinic, type, done, notes } ],
      clinicalRecords: [ { id, type, date, week,
        result, status } ],
      symptomLogs: [ { date, tags[], notes } ],
      kickSessions: [ { date, kicks,
        durationMinutes } ],
      waterGlasses, waterDate,
      doctorQuestions: [ { id, question, asked } ],
      forumPosts: [ { id, club, author, title,
        replies, likes } ],
      checklists: [ { id, cat, label, done } ],
      chatMessages: [ { role, text, timestamp } ],
      settings: { language, geminiApiKey,
        groqApiKey, timelineView }
    }
  }
}
```

Document Collections (arrays per user)

Collection	Key Fields	Purpose
profile	name, status, edd, city, doctorName, ec_phone	Core identity & pregnancy status
appointments	id, title, date, time, doctor, clinic, done	Checkups & scans
clinicalRecords	id, type, date, week, result, status	Lab / scan reports

symptomLogs	date, tags[], notes	Daily symptom tracking
kickSessions	date, kicks, durationMinutes	Baby kick counter
doctorQuestions	id, question, asked	Questions to ask at next visit
checklists	id, cat, label, done	Hospital bag / baby / postpartum prep
chatMessages	role, text, timestamp	AI chat history (last 100 kept)
settings	language, geminiApiKey, groqApiKey	Per-user app preferences

REST API Endpoints

Method	Endpoint	Purpose
GET	/api/db/status	Database health & record counts
GET	/api/records/all	Fetch all data for a user
POST	/api/records/sync	Batch sync from frontend
POST	/api/user/profile	Update profile
GET/POST/PUT/DELETE	/api/appointments/:id	Appointment CRUD
GET/POST/DELETE	/api/clinical-records/:id	Clinical record CRUD
POST	/api/symptoms	Add symptom log
POST	/api/kicks	Add kick session
POST	/api/chat	AI chat (Gemini / Groq)
GET	/api/export	Download full JSON backup
POST	/api/import	Restore from JSON backup